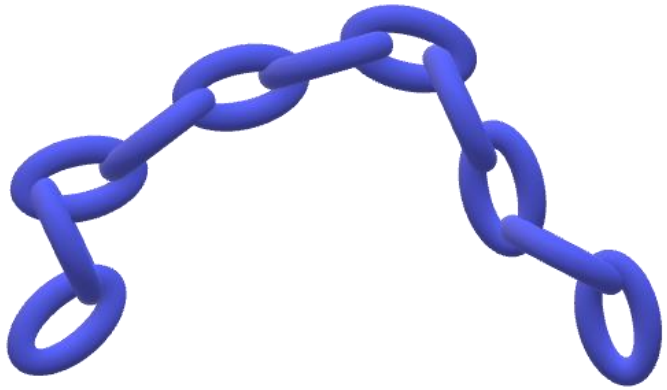
ESTRUTURAS DE DADOS BÁSICAS PARTE 2/3

Alocação Encadeada



Alocação Encadeada

Variável do tipo ponteiro

Conteúdo é um endereço de memória

Alocação Encadeada

Variável do tipo ponteiro

Nome ← “Maria”

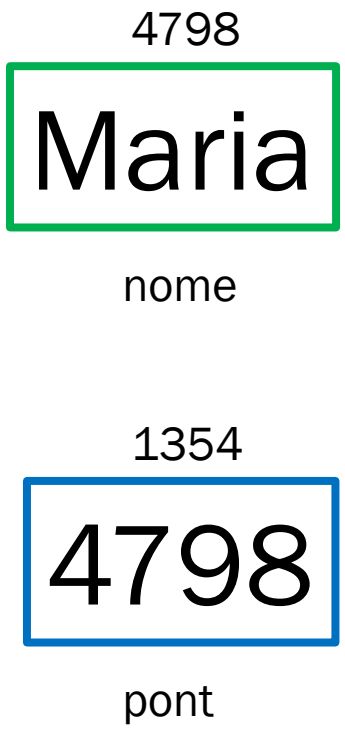


Alocação Encadeada

Variável do tipo ponteiro

nome ← “Maria”

pont ← &nome



Alocação Encadeada

Variável do tipo ponteiro

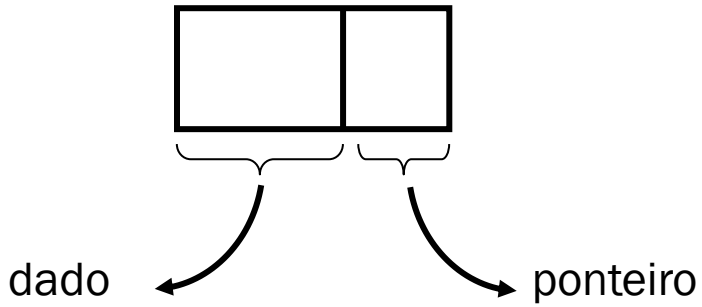


Ponteiro

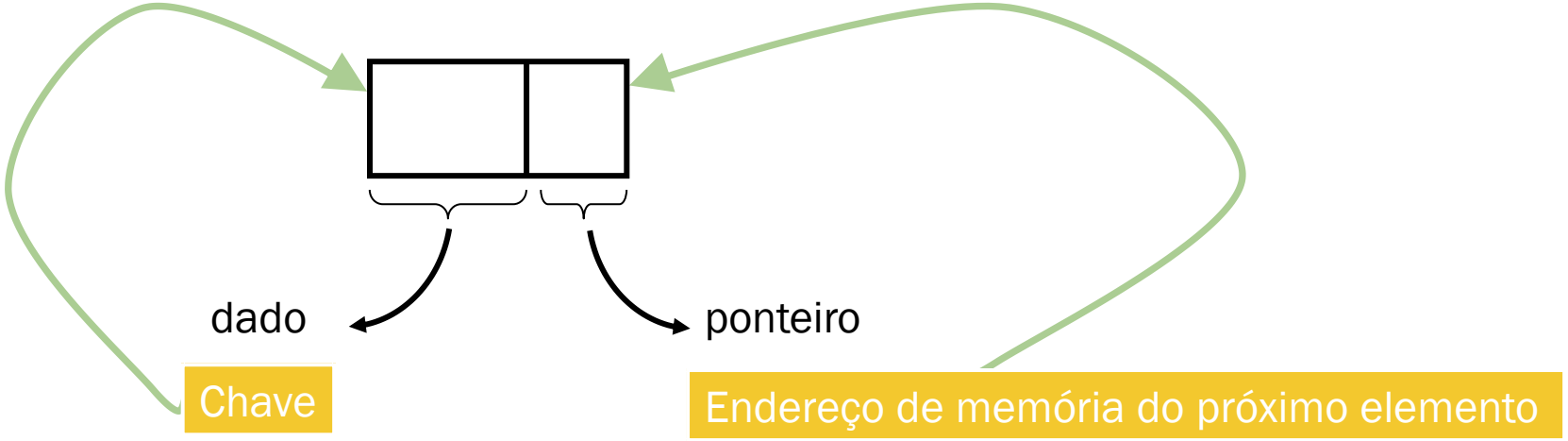
Um ponteiro é uma variável que guarda o endereço de outra variável.

O valor da variável ponteiro pode ser nulo (“null”) significando que não está guardando o endereço de outra variável.

Alocação Encadeada



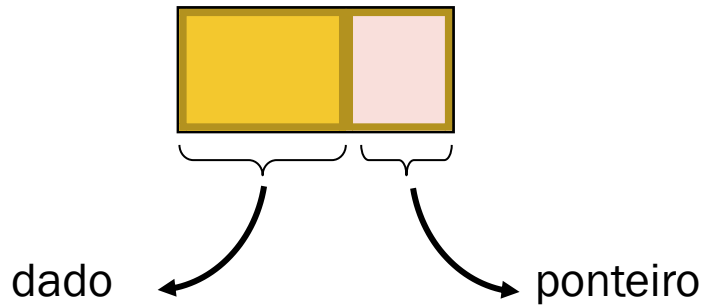
Alocação Encadeada



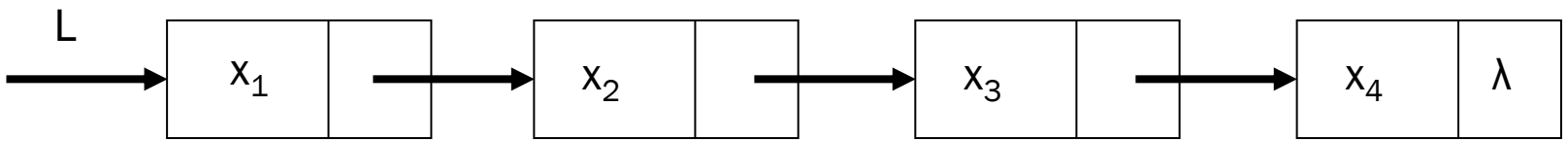
Alocação Encadeada

- Posições na memória são alocadas ou desalocadas na medida em que são necessárias.

- Os nós da lista encontram-se em posições quaisquer na memória e são interligados por ponteiros que indicam a posição do próximo elemento da lista.

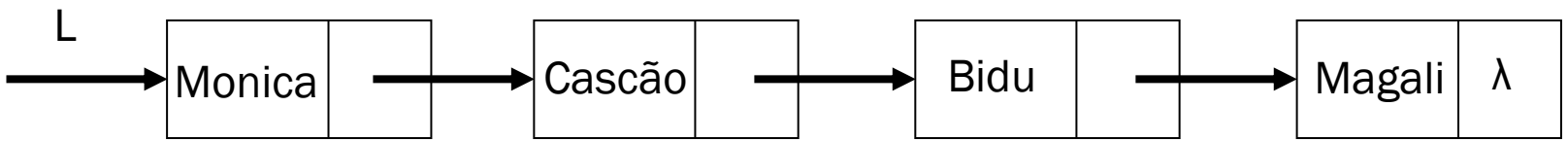


Lista Linear Simplesmente Encadeada



Obs. L é um ponteiro que guarda o endereço de memória do primeiro elemento da lista.

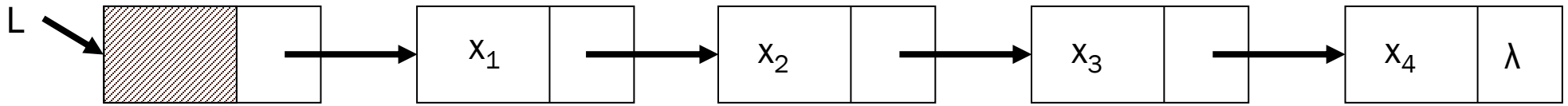
Lista Linear Simplesmente Encadeada



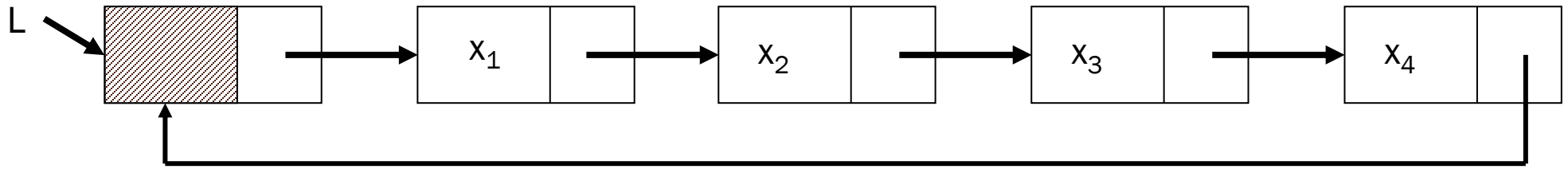
Obs. L é um ponteiro que guarda o endereço de memória do primeiro elemento da lista.

Lista Linear Simplesmente Encadeada

Exemplo 1 (com nó cabeça)



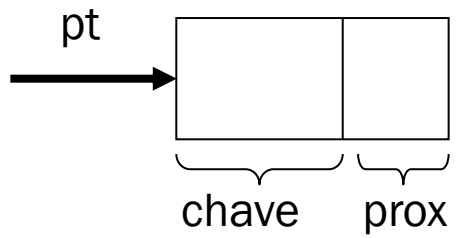
Exemplo 2 (circular com nó cabeça)



Criação de uma posição na memória

Supor que o ponteiro aponta para um tipo de variável que tem 2 campos: *chave* (dado) e *prox* (ponteiro para outra variável do mesmo tipo).

Comando:
ocupar(pt)



Efeito: *a variável pt recebe o endereço de memória de um elemento composto de dois campos: chave e prox*

Liberação de uma posição na memória

Comando:
desocupar(pt)

Efeito: *a posição na memória não está mais referenciada pela variável pt.*



Busca em Lista Ordenada

Busca(x,ant,pont) //lista simplesmente encadeada com nó cabeça

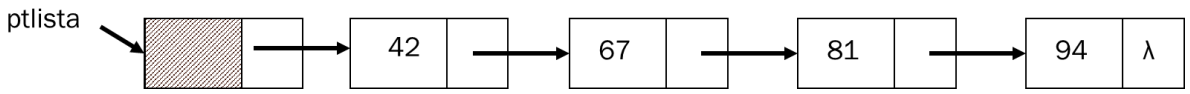
```
ant←ptlista; pont←λ
ptaux ← ptlista↑.prox
enquanto(pt_aux ≠λ) faça
  se (pt_aux↑.chave < x) então
      ant ← pt_aux
      pt_aux ← pt_aux↑.prox
  senão
      se (pt_aux↑.chave = x) então
          pont ← pt_aux
      pt_aux ← λ
```

Complexidade ?

Busca em Lista Ordenada

Busca(x,ant,pont) //lista simplesmente encadeada com nó cabeça

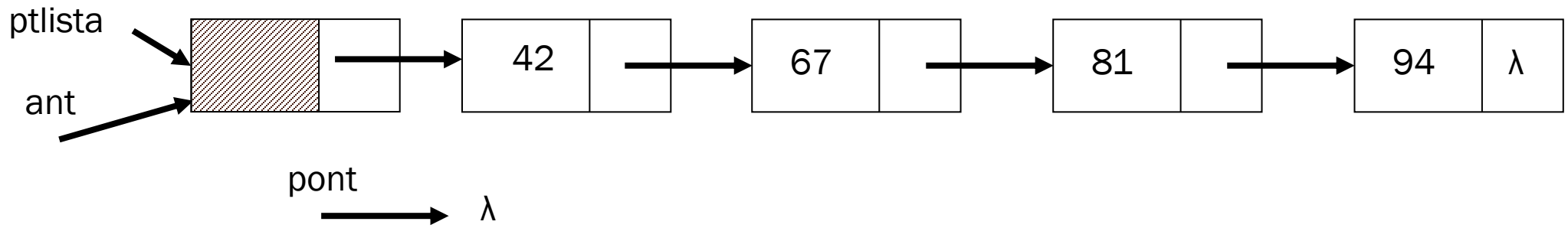
```
ant←ptlista; pont←λ
ptaux ← ptlista↑.prox
enquanto(pt_aux ≠λ) faça
   se (pt_aux↑.chave < x) então
        ant ← pt_aux
        pt_aux ← pt_aux↑.prox
   senão
        se (pt_aux↑.chave = x) então
             pont ← pt_aux
        pt_aux ← λ
```



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

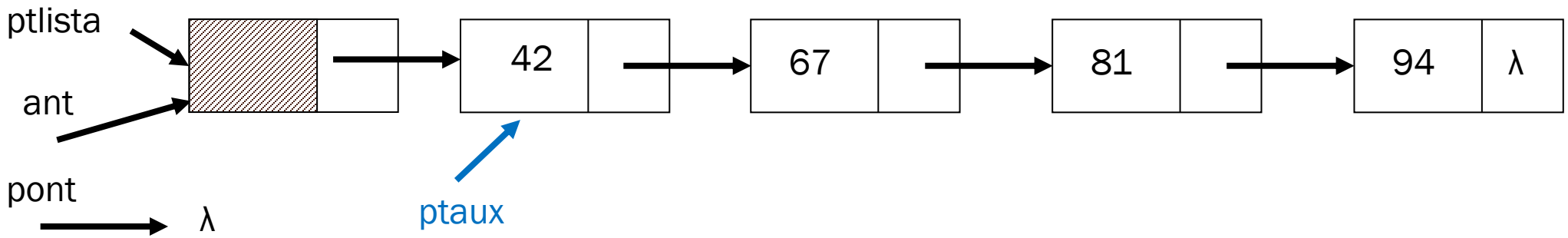
Exemplo: x=81



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

Exemplo: x=81

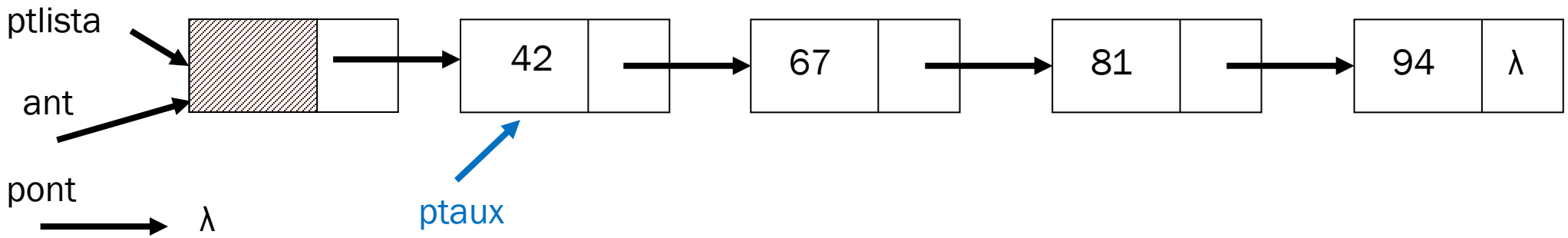


Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

OK

Exemplo: x=81



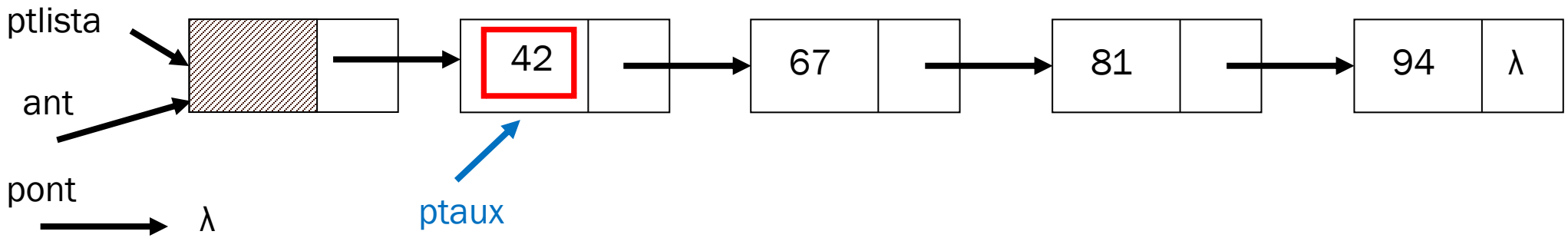
Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

OK

42 < 81

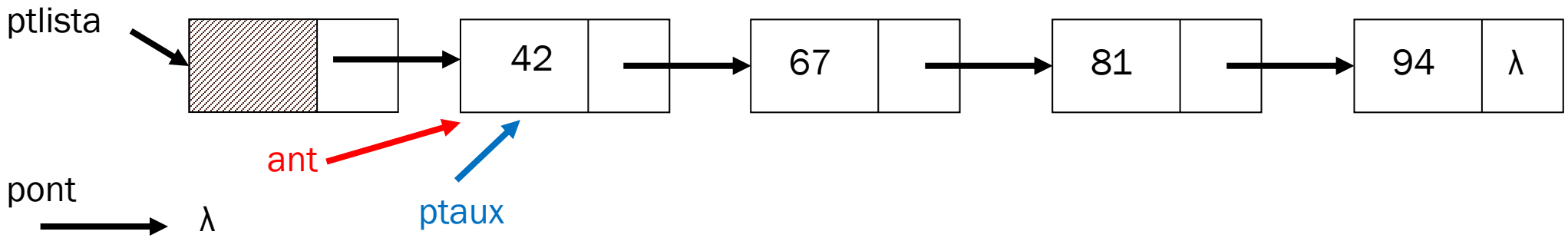
Exemplo: x=81



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

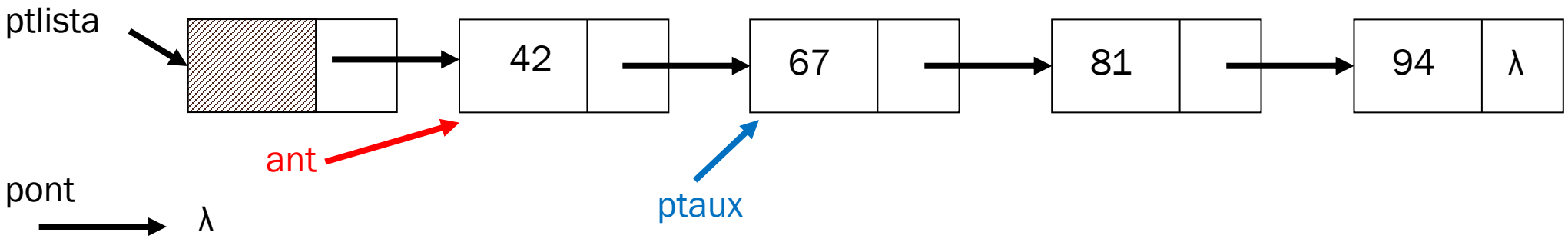
Exemplo: x=81



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

Exemplo: x=81

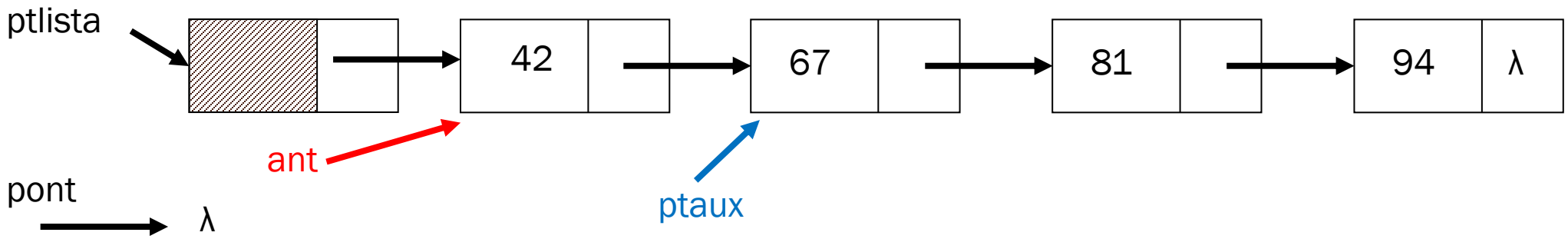


Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

OK

Exemplo: x=81



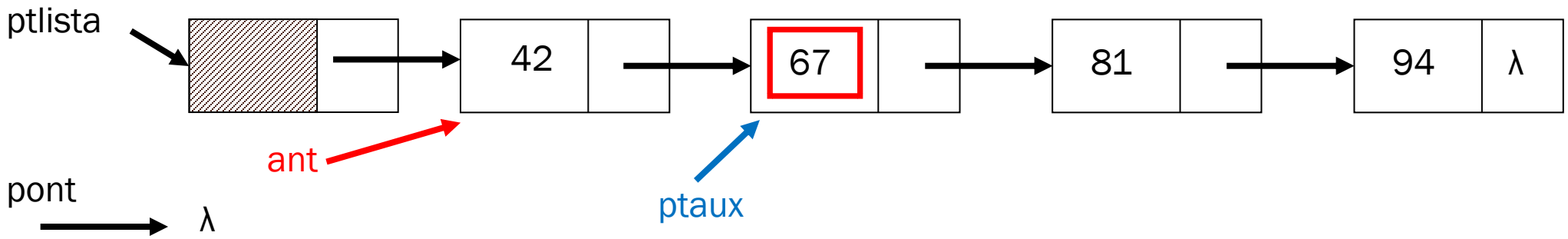
Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

OK

67 < 81

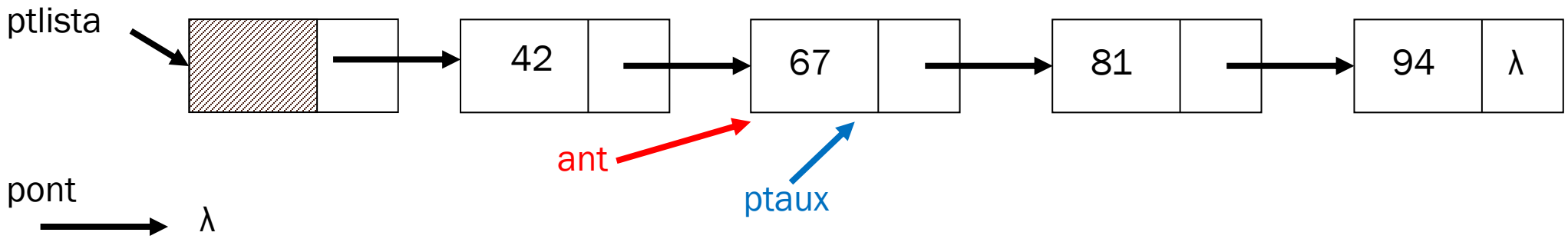
Exemplo: x=81



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

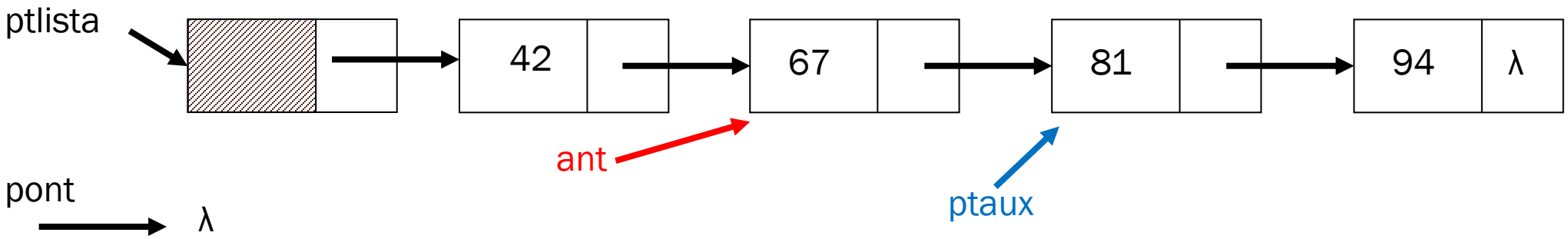
Exemplo: x=81



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

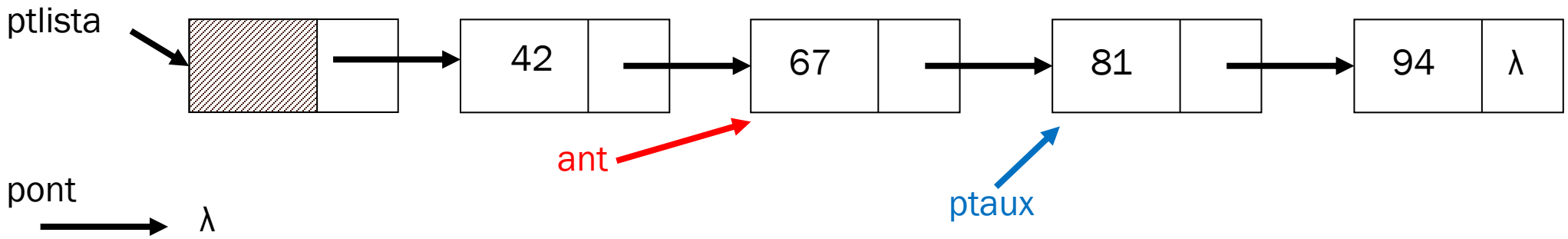
Exemplo: x=81



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

Exemplo: x=81

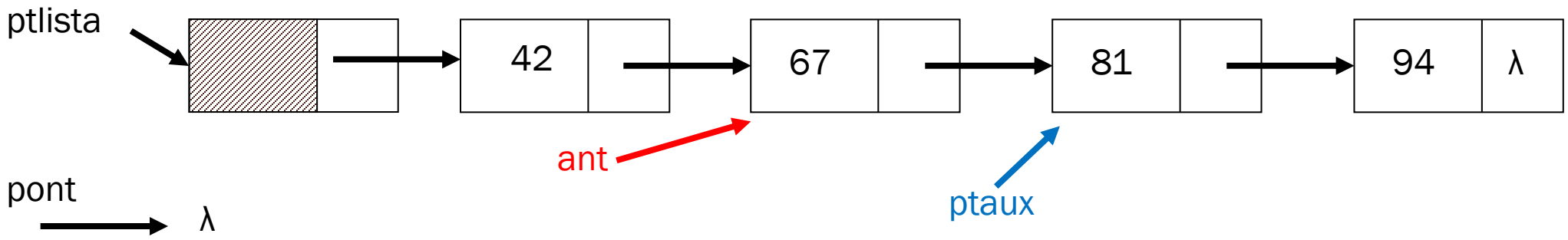


Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

81 = 81

Exemplo: x=81

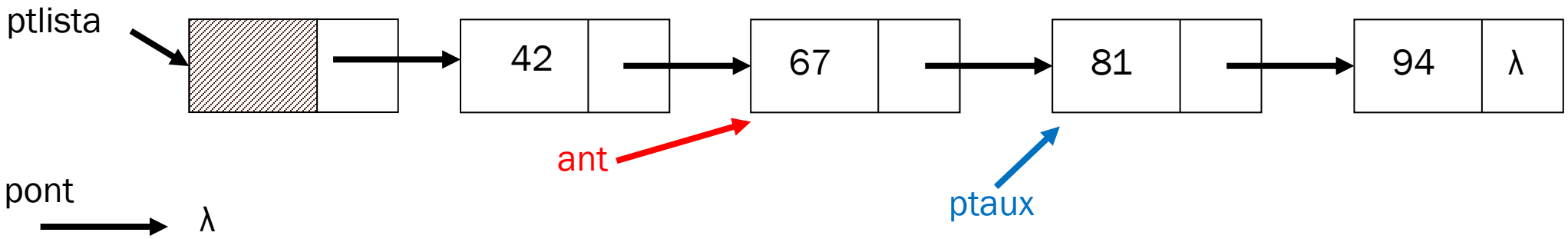


Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
            pt_aux ← λ
```

OK

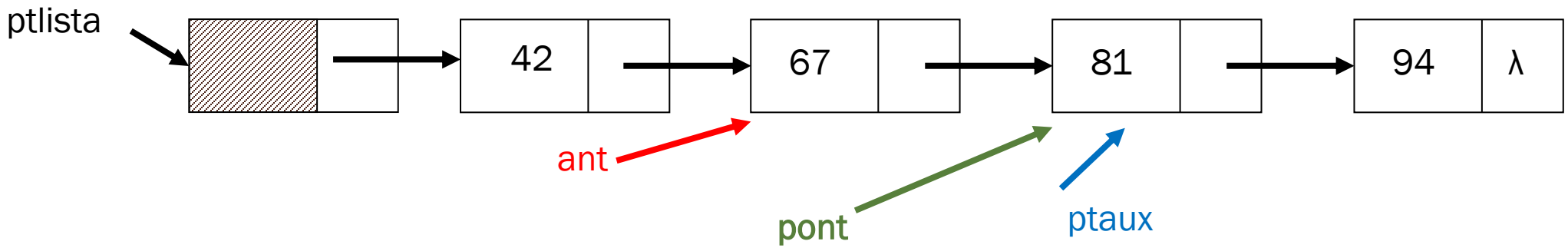
Exemplo: x=81



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

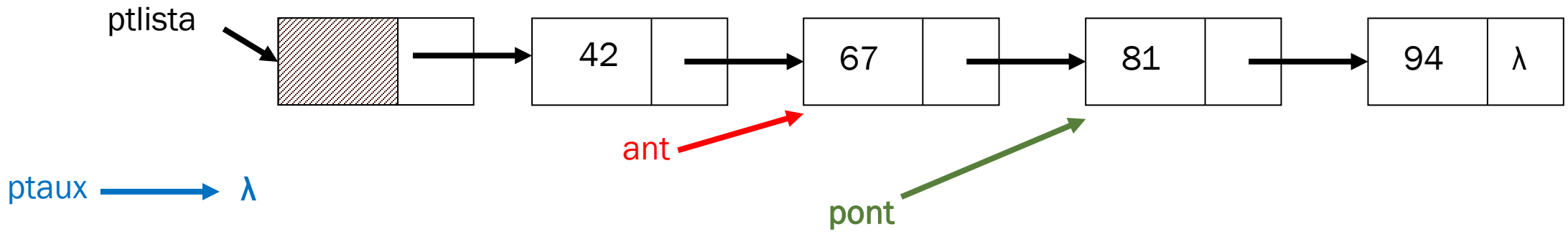
Exemplo: x=81



Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

Exemplo: x=81

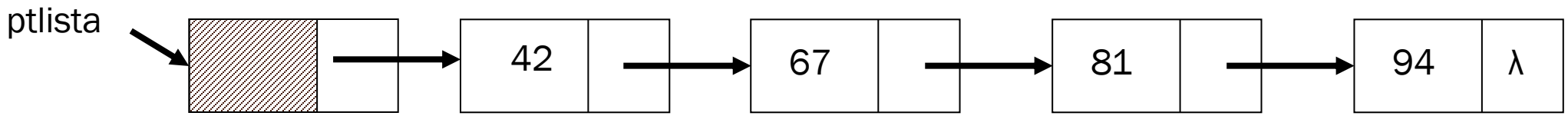


Busca em Lista Ordenada

```
Busca(x,ant,pont)
    ant←ptlista; pont←λ
    ptaux ← ptlista↑.prox
    enquanto(pt_aux ≠λ) faça
      se (pt_aux↑.chave < x) então
          ant ← pt_aux
          pt_aux ← pt_aux↑.prox
      senão
          se (pt_aux↑.chave = x) então
              pont ← pt_aux
          pt_aux ← λ
```

O que acontece se a chave de busca for x=70?

EXERCÍCIO



Busca em Lista Ordenada

Busca(x,ant,pont) //lista simplesmente encadeada com nó cabeça

```
ant←ptlista; pont←λ
ptaux ← ptlista↑.prox
enquanto(pt_aux ≠λ) faça
  se (pt_aux↑.chave < x) então
      ant ← pt_aux
      pt_aux ← pt_aux↑.prox
  senão
      se (pt_aux↑.chave = x) então
          pont ← pt_aux
      pt_aux ← λ
```

COMPLEXIDADE?

Busca em Lista Ordenada

Busca(x,ant,pont) //lista simplesmente encadeada com nó cabeça

```
ant←ptlista; pont←λ
ptaux ← ptlista↑.prox
enquanto(pt_aux ≠λ) faça
   se (pt_aux↑.chave < x) então
        ant ← pt_aux
        pt_aux ← pt_aux↑.prox
   senão
        se (pt_aux↑.chave = x) então
             pont ← pt_aux
        pt_aux ← λ
```

Melhor caso

Encontra a chave de busca na primeira posição
ou
Chave de busca é menor que a chave do primeiro elemento da lista

$\Omega(1)$

Busca em Lista Ordenada

Busca(x,ant,pont) //lista simplesmente encadeada com nó cabeça

```
ant←ptlista; pont←λ
ptaux ← ptlista↑.prox
enquanto(pt_aux ≠λ) faça
   se (pt_aux↑.chave < x) então
        ant ← pt_aux
        pt_aux ← pt_aux↑.prox
   senão
        se (pt_aux↑.chave = x) então
             pont ← pt_aux
        pt_aux ← λ
```

Pior caso

Encontra a chave de busca na última posição

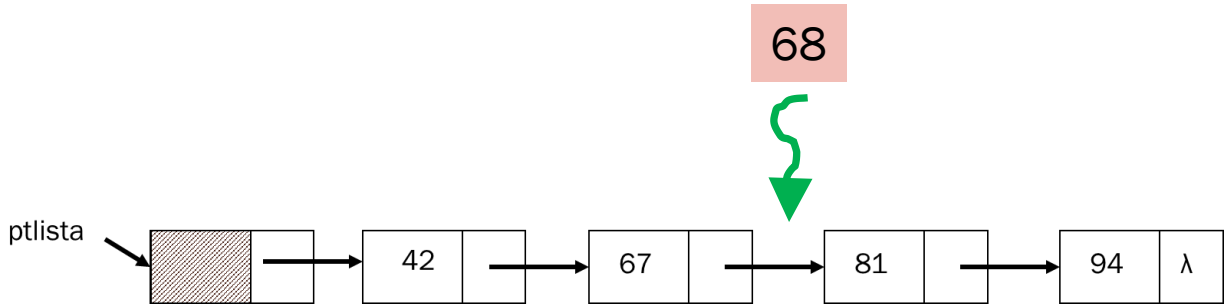
ou

Chave de busca é maior que a chave do último elemento da lista

$O(n)$

Inserção

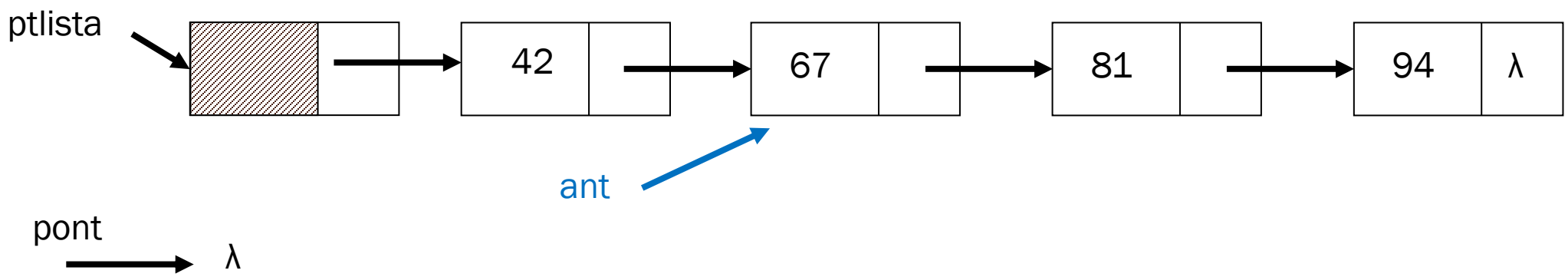
```
Insere(x)
    Busca(x,ant,pont)
    se (pont = λ) então
        ocupar(pt)
        pt↑.chave ← x
        pt↑.prox ← ant↑.prox
        ant↑.prox ← pt
    senão “Elemento já se
encontra na lista”
```



Inserção

```
Insere(x)
    Busca(x,ant,pont)
    se (pont = λ) então
        ocupar(pt)
        pt↑.chave ← x
        pt↑.prox ← ant↑.prox
        ant↑.prox ← pt
    senão “Elemento já se
encontra na lista”
```

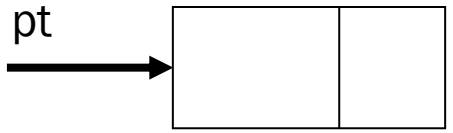
Exemplo: x=68

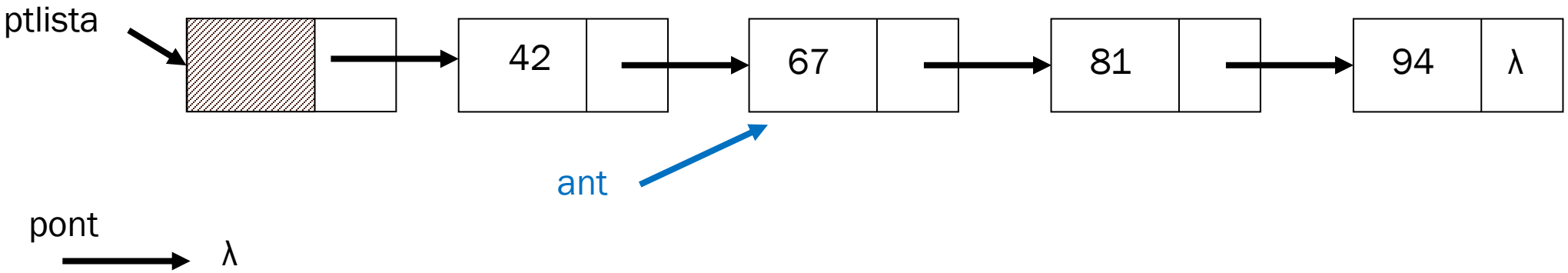


Inserção

```
Insere(x)
    Busca(x,ant,pont)
    se (pont = λ) então
        ocupar(pt)
        pt↑.chave ← x
        pt↑.prox ← ant↑.prox
        ant↑.prox ← pt
    senão "Elemento já se
encontra na lista"
```

Exemplo: x=68

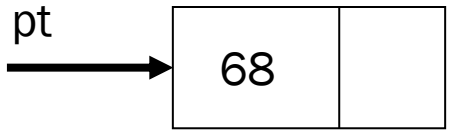


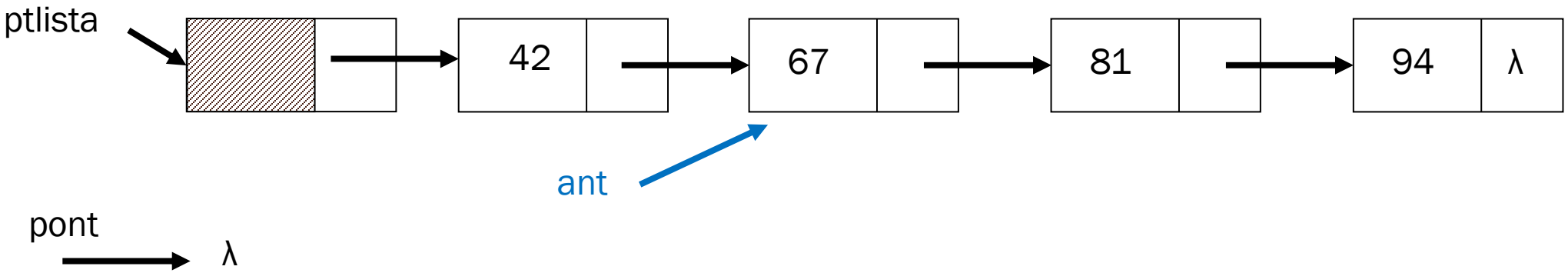


Inserção

```
Insere(x)
    Busca(x,ant,pont)
    se (pont = λ) então
        ocupar(pt)
        pt↑.chave ← x
        pt↑.prox ← ant↑.prox
        ant↑.prox ← pt
    senão "Elemento já se
encontra na lista"
```

Exemplo: x=68

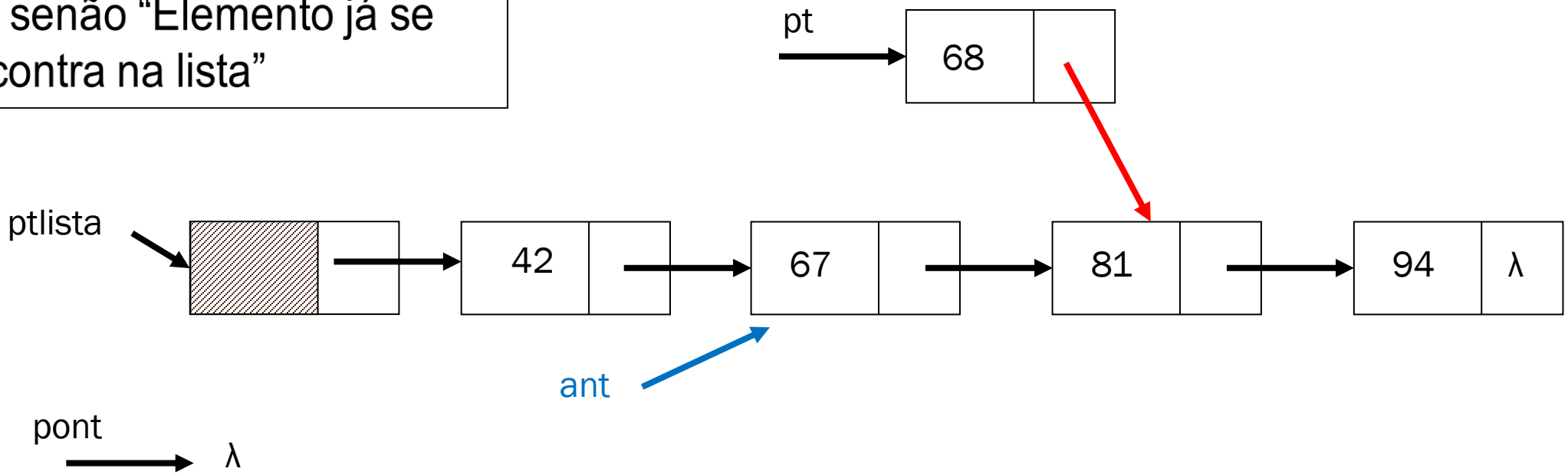




Inserção

```
Insere(x)
    Busca(x,ant,pont)
    se (pont = λ) então
        ocupar(pt)
        pt↑.chave ← x
        pt↑.prox ← ant↑.prox
        ant↑.prox ← pt
    senão “Elemento já se
encontra na lista”
```

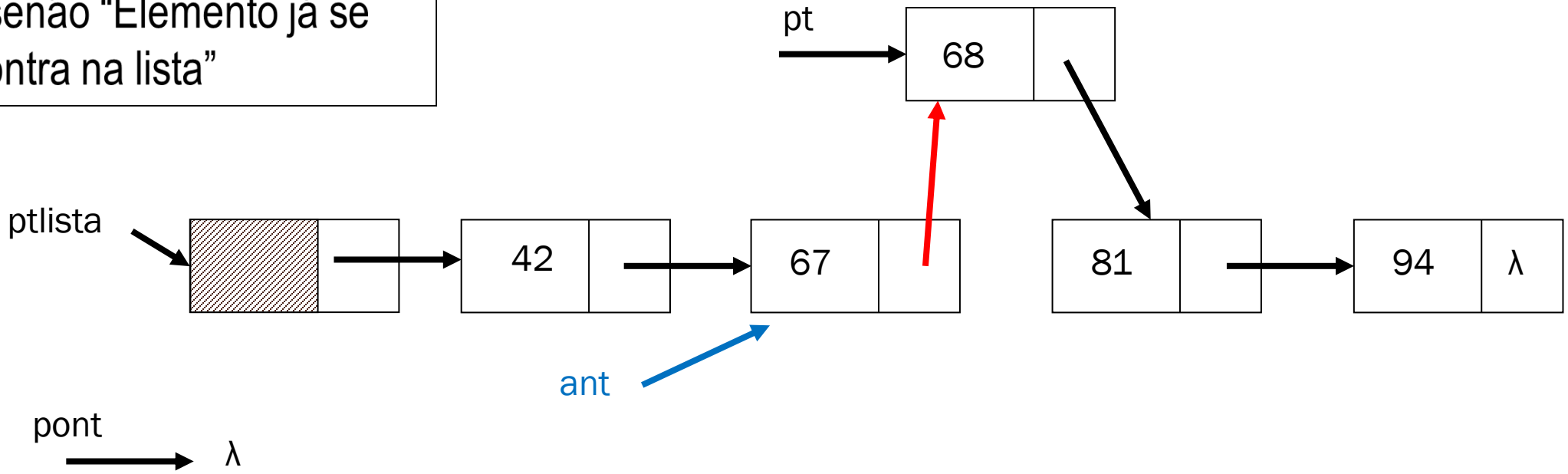
Exemplo: x=68



Inserção

```
Insere(x)
    Busca(x,ant,pont)
    se (pont = λ) então
        ocupar(pt)
        pt↑.chave ← x
        pt↑.prox ← ant↑.prox
        ant↑.prox ← pt
    senão "Elemento já se
encontra na lista"
```

Exemplo: x=68



Inserção

```
Insere(x)
      Busca(x,ant,pont)
      se (pont = λ) então
          ocupar(pt)
          pt↑.chave ← x
          pt↑.prox ← ant↑.prox
          ant↑.prox ← pt
      senão “Elemento já se
encontra na lista”
```

Complexidade ?

Inserção

```
Insere(x)
    Busca(x,ant,pont)
    se (pont = λ) então
        ocupar(pt)
        pt↑.chave ← x
        pt↑.prox ← ant↑.prox
        ant↑.prox ← pt
    senão “Elemento já se
encontra na lista”
```

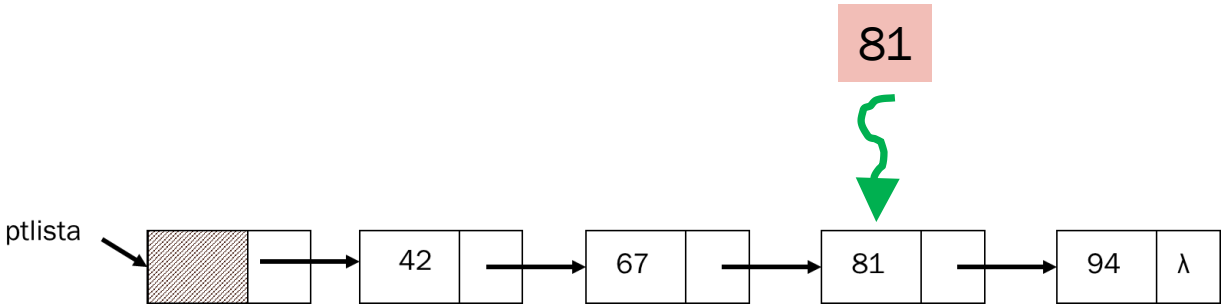
Busca é $\Omega(1)$ e $O(n)$

Demais comandos: $\theta(1)$

Complexidade é $\Omega(1)$ e $O(n)$

Remoção

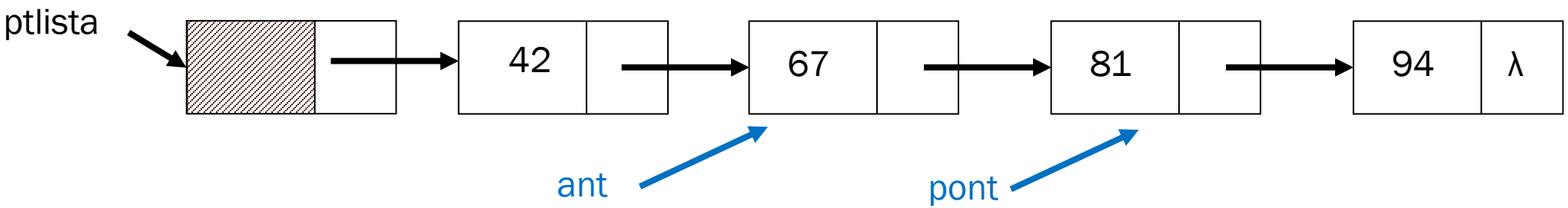
```
Remove(x)
    Busca(x,ant,pont)
    se (pont ≠ λ) então
        ant↑.prox ← pont↑.prox
        desocupar(pont)
    senão “Elemento não se
encontra na lista”
```



Remoção

```
Remove(x)
    Busca(x,ant,pont)
    se (pont ≠ λ) então
        ant↑.prox ← pont↑.prox
        desocupar(pont)
    senão “Elemento não se
encontra na lista”
```

Exemplo: x=81

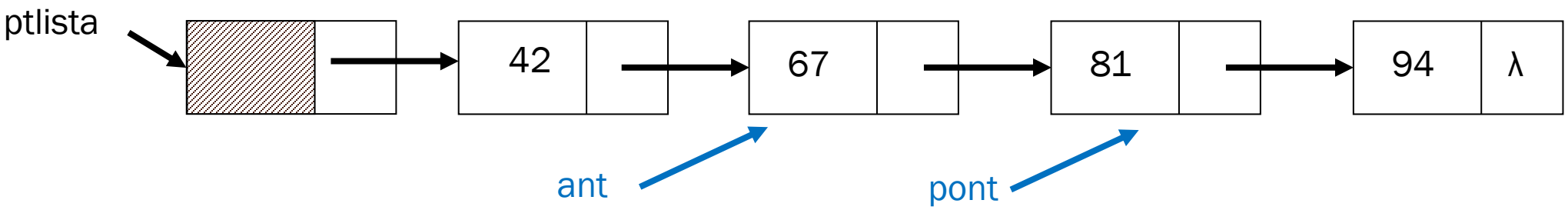


Remoção

```
Remove(x)
    Busca(x,ant,pont)
    se (pont ≠ λ) então
        ant↑.prox ← pont↑.prox
        desocupar(pont)
    senão "Elemento não se
encontra na lista"
```

OK

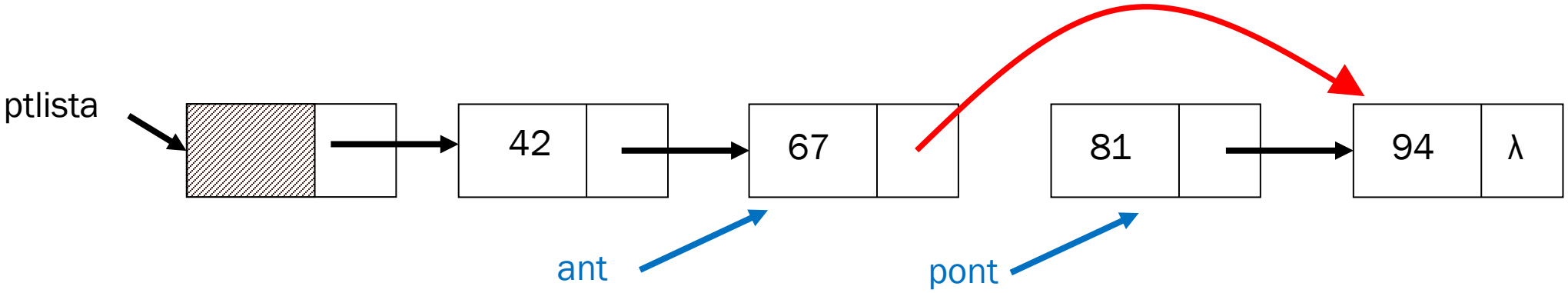
Exemplo: x=81



Remoção

```
Remove(x)
    Busca(x,ant,pont)
    se (pont ≠ λ) então
        ant↑.prox ← pont↑.prox
        desocupar(pont)
    senão "Elemento não se
encontra na lista"
```

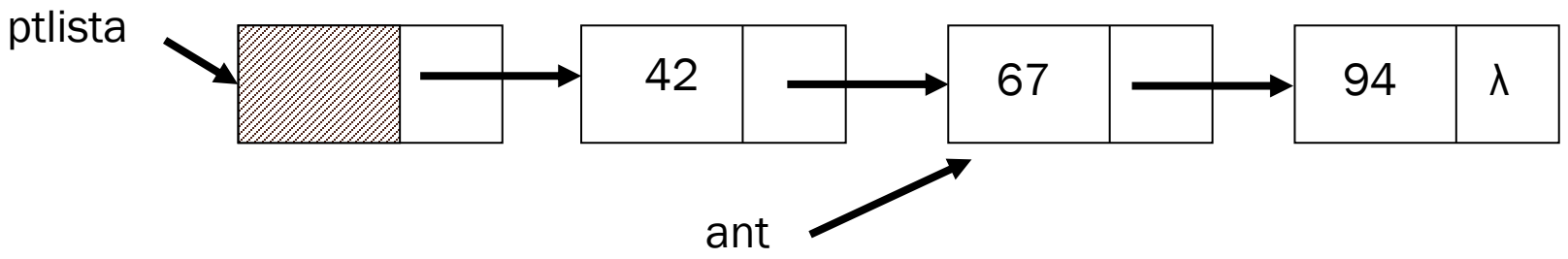
Exemplo: x=81



Remoção

```
Remove(x)
    Busca(x,ant,pont)
    se (pont ≠ λ) então
        ant↑.prox ← pont↑.prox
        desocupar(pont)
    senão "Elemento não se
encontra na lista"
```

Exemplo: x=81



Remoção

```
Remove(x)
      Busca(x,ant,pont)
      se (pont ≠ λ) então
           ant↑.prox ← pont↑.prox
          desocupar(pont)
      senão “Elemento não se
encontra na lista”
```

Complexidade ?

Remoção

```
Remove(x)
      Busca(x,ant,pont)
      se (pont ≠ λ) então
            ant↑.prox ← pont↑.prox
           desocupar(pont)
      senão “Elemento não se
encontra na lista”
```

Busca é $\Omega(1)$ e $O(n)$

Demais comandos: $\theta(1)$

Complexidade é $\Omega(1)$ e $O(n)$